

部署即代码：应用与基础设施全面代码化

2026-09-24 14:56

目录

一、为什么需要“部署即代码”

二、核心理念：把交付过程纳入软件研发范式

三、到底哪些东西应该代码化

3.1 应用代码化：Application / ApplicationSet

3.2 基础设施代码化：Terraform Layer / LayerSet

3.3 环境代码化：目录结构就是环境拓扑

3.4 交付策略代码化：不仅描述“部署什么”，还描述“怎么部署”

四、PowerAPP 推荐的整体交付模型

五、应用与基础设施如何协同

六、实现方式：ApplicationSet + multi-source + Git Generator

6.1 Chart 与 values 分离

6.2 ApplicationSet 自动发现环境文件

七、版本发布与变更流程

八、这种模式带来的核心收益

8.1 可追溯

8.2 可复现

8.3 可评审

8.4 可扩展

8.5 可治理

8.6 活文档：Git 仓库就是业务运行态的说明书

九、最佳实践

9.1 目录优先，分支辅助

9.2 生产环境必须走 MR

9.3 Chart 和 values 分离

9.4 应用和基础设施分层，但流程统一

9.5 高风险动作默认保守

9.6 把仓库当成第一入口来维护

十、常见误区

十一、可参考的 Demo

十二、相关文档

部署即代码 (Deployment as Code) 的核心不是“把 YAML 放进 Git”，而是把一次应用交付所依赖的全部事实——应用版本、运行参数、环境差异、集群目标、基础设施、密钥引用、同步策略、检查规则、发布流程——都用代码描述，并通过 Git / MR / Review / 自动化检查 / 持续同步来管理。

最终目标是：**应用怎么研发，部署和运维就怎么管理。**

- 需求变更要写代码，部署变更也写代码；
- 业务代码要评审，环境配置和基础设施也要评审；
- 代码可以版本化、回滚、追溯，部署与基础设施也应该一样；
- 软件研发依赖 CI/CD 自动化，环境交付也应该由平台持续收敛，而不是人工登录控制台逐项操作。

PowerAPP 正是围绕这个理念建设：通过 GitOps 把**应用交付与基础设施管理**统一到同一种工程化模式中。

一、为什么需要“部署即代码”

在传统模式下，一个应用上线往往不只是“部署一个镜像”这么简单，还依赖一整套环境事实：

- Kubernetes 资源：Deployment / Service / Ingress / ConfigMap / Secret / HPA ；
- 应用参数：镜像版本、副本数、资源规格、启动参数、灰度策略；
- 基础设施：CLB、DNS、VPC、安全组、数据库、Redis、对象存储、消息队列；
- 环境映射：哪个环境部署到哪个集群、哪个 namespace、哪个地域；
- 运维策略：同步策略、健康检查、告警、回滚、权限、审计。

如果这些信息分散在不同平台、不同人员脑子里、不同控制台的手工配置里，就会出现典型问题：

问题	传统表现	影响
环境不可复现	测试环境和生产环境靠人工配置	同一版本在不同环境表现不一致
变更不可追溯	控制台直接修改，无 commit / MR	出问题后不知道谁改了什么
配置容易漂移	Git 一套，集群一套，云资源又一套	线上实际状态偏离预期
部署文档容易失效	文档写完后没人维护，实际环境已变化	新人按文档操作失败，排障时拿到的是过期信息
扩容成本高	新增环境 / 新增集群要复制粘贴配置	规模越大越容易出错
协作成本高	研发、SRE、平台团队通过工单来回确认	上线链路长，责任边界模糊

“部署即代码”解决的正是这些问题：**把环境和部署也纳入软件工程体系**，让它们像业务代码一样被管理。

二、核心理念：把交付过程纳入软件研发范式

软件研发有一套成熟范式：

1 | 需求 → 代码修改 → 本地/CI 检查 → MR Review → 合并 → 自动构建/发布 → 监控验证

部署即代码希望把应用与基础设施交付也变成同样的模式：

1 | 环境/部署需求 → 修改 Git 配置 → Diff / DryRun / Plan → MR Review → 合并 → PowerAPP 持续同步 → 健康检查 / 漂移治理

二者的对应关系如下：

软件研发	部署即代码 / GitOps
业务代码	Helm values / K8s YAML / Terraform 代码 / ApplicationSet 模板
编译检查	YAML Lint / Helm Template / K8s DryRun / Terraform Plan
Code Review	MR Review：评审环境参数、资源规格、基础设施变更
CI/CD	PowerAPP + Argo CD + Terraform Controller 持续收敛
Git Commit	每次部署、基础设施变更的审计记录
Release Tag	固定应用版本、Chart 版本、Terraform Module 版本
Rollback	回退 Git commit / tag，平台自动把环境收敛回旧状态

所以，部署即代码不是额外增加一套流程，而是把部署和运维纳入团队本来就熟悉的研发协作流程中。

三、到底哪些东西应该代码化

一个完整环境至少包含三层：应用、基础设施、交付策略。PowerAPP 希望这三层都能代码化。

3.1 应用代码化：Application / ApplicationSet

应用层描述“我的服务如何运行”：

- 使用哪个 Git 仓库、Chart、目录或镜像版本；
- 部署到哪个集群、哪个 namespace；
- 使用哪些 values 参数；
- 副本数、资源规格、探针、HPA、环境变量；
- 同步策略、忽略差异规则、健康检查和回滚策略。

PowerAPP 中通常由两类资源承载：

资源	用途
Application	管理单个应用的 GitOps 部署
ApplicationSet	按环境、集群、目录批量生成多个 Application

当业务只有一个服务、一个环境时，Application 就足够；当业务进入多环境、多集群、多地域阶段，应使用 ApplicationSet，把“环境矩阵”也代码化。

3.2 基础设施代码化：Terraform Layer / LayerSet

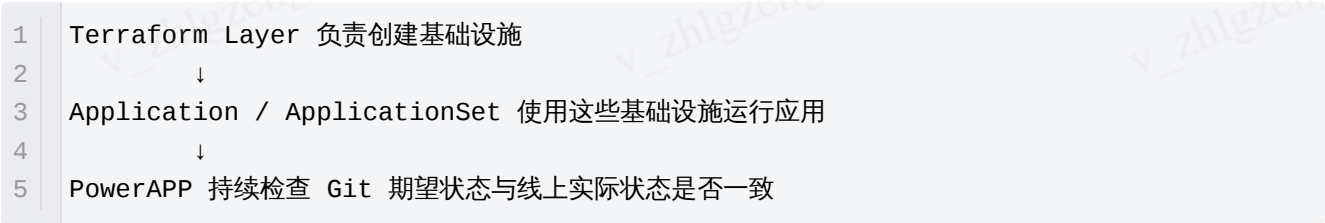
应用往往依赖云资源，例如：

- 负载均衡 CLB / 监听器 / 转发规则；
- DNS 解析、证书、域名；
- VPC、子网、安全组；
- 数据库、Redis、COS、消息队列；
- 跨地域、跨云厂商资源。

这些不应该长期依赖人工点控制台创建。PowerAPP 通过 Terraform 能力把基础设施也纳入 GitOps：

资源	用途
Terraform Layer	单个 Terraform 部署单元，负责一组云资源
Terraform LayerSet	批量生成多个 Layer，适合多地域 / 多环境基础设施
Terraform Modules	沉淀可复用的基础设施模块

这样，应用和基础设施就不再是两套割裂系统，而是同一套 GitOps 交付模型中的两个层次：



3.3 环境代码化：目录结构就是环境拓扑

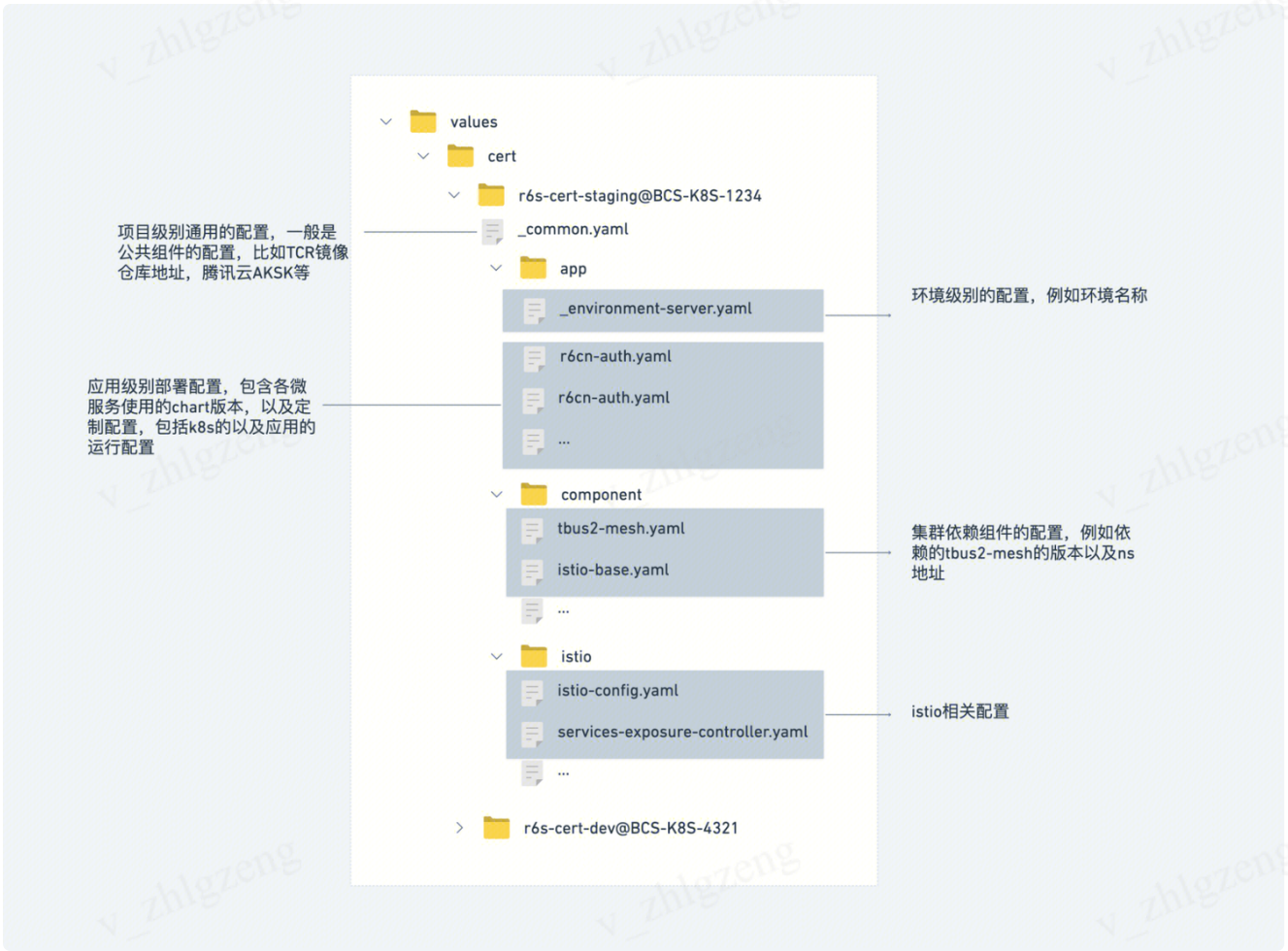
环境不应该只存在于口头约定中。建议通过 Git 目录明确表达：

```
1 gitops-values/  
2 |— values-cn/  
3 |   |— dev/  
4 |   |   |— dev-cluster@BCS-K8S-00001/app/frontend.yaml  
5 |   |— pre/  
6 |   |   |— pre-cluster@BCS-K8S-00002/app/frontend.yaml  
7 |   |— prod/  
8 |   |   |— prod-cluster@BCS-K8S-00003/app/frontend.yaml  
9 |— values-sg/  
10 |   |— prod/  
11 |       |— sg-cluster@BCS-K8S-00010/app/frontend.yaml
```

这棵目录树表达了三个事实：

1. 环境：dev / pre / prod；
2. 集群：prod-cluster@BCS-K8S-00003；
3. 应用实例：app/frontend.yaml。

新增一个应用 = 新增一个 values 文件；新增一个环境 = 新增一个目录；环境迁移到新集群 = 调整目录中的集群映射。



3.4 交付策略代码化：不仅描述“部署什么”，还描述“怎么部署”

除了资源本身，还应把交付策略写进代码：

- 同步策略：手动同步、自动同步、是否允许删除；
- 安全策略：生产分支保护、MR 审核、Terraform 安全组检查；
- 发布策略：金丝雀、蓝绿、分批同步；
- 差异策略：哪些字段允许临时人工修改，哪些必须由 Git 校准；
- 回收策略：删除 Application / Layer 时是否删除真实资源。

这些策略如果只靠人工记忆，很容易在压力环境下失效；写进代码后，平台才能持续执行。

四、PowerAPP 推荐的整体交付模型

推荐把仓库分为三类：

仓库	维护者	内容
业务代码仓库	研发团队	源码、Dockerfile、单元测试、构建流水线

Chart / 模板仓库	平台 / SRE / 研发共建	Helm Chart、Kustomize base、通用模板
Values / 环境仓库	SRE / 业务运维	各环境差异、集群映射、应用参数
Terraform 仓库	SRE / 基础设施团队	CLB、DNS、DB、Redis、VPC、安全组等 IaC 配置

这些 Git 仓库不只是“部署输入”，也是业务的**活文档**：它们真实记录了当前线上环境长什么样、每个应用部署到哪里、依赖哪些云资源、为什么这样配置，以及历史上每一次变更是如何发生的。

交付链路如下：

1	业务代码变更
2	→ 构建镜像 / 生成版本
3	→ 修改 values 或发布配置
4	→ MR Review + 自动检查
5	→ 合并到目标环境分支
6	→ PowerAPP 自动感知 Git 变更
7	→ Argo CD 同步应用，Terraform Controller 同步基础设施
8	→ 健康检查 / Diff / Drift Detection / AI 根因分析

重点是：**真正驱动线上变化的是 Git 合并**，而不是某个人在平台上点了一个不可追溯的按钮。

五、应用与基础设施如何协同

以一个游戏网关服务为例，完整交付可能包括：

层次	代码化内容	PowerAPP 能力
基础网络	VPC、子网、安全组	Terraform Layer
流量入口	CLB、监听器、DNS、证书	Terraform Layer / Module
运行环境	集群、namespace、label、quota	Application / ApplicationSet
应用工作负载	Deployment、Service、ConfigMap、HPA	Application
多环境编排	dev/pre/prod 的 values、集群映射	ApplicationSet Git Generator
发布治理	同步策略、分支保护、Review、Diff、DryRun	PowerAPP GitOps 流程
运维排障	Events、日志、资源拓扑、AI 分析	AI 小鲸 / MCP

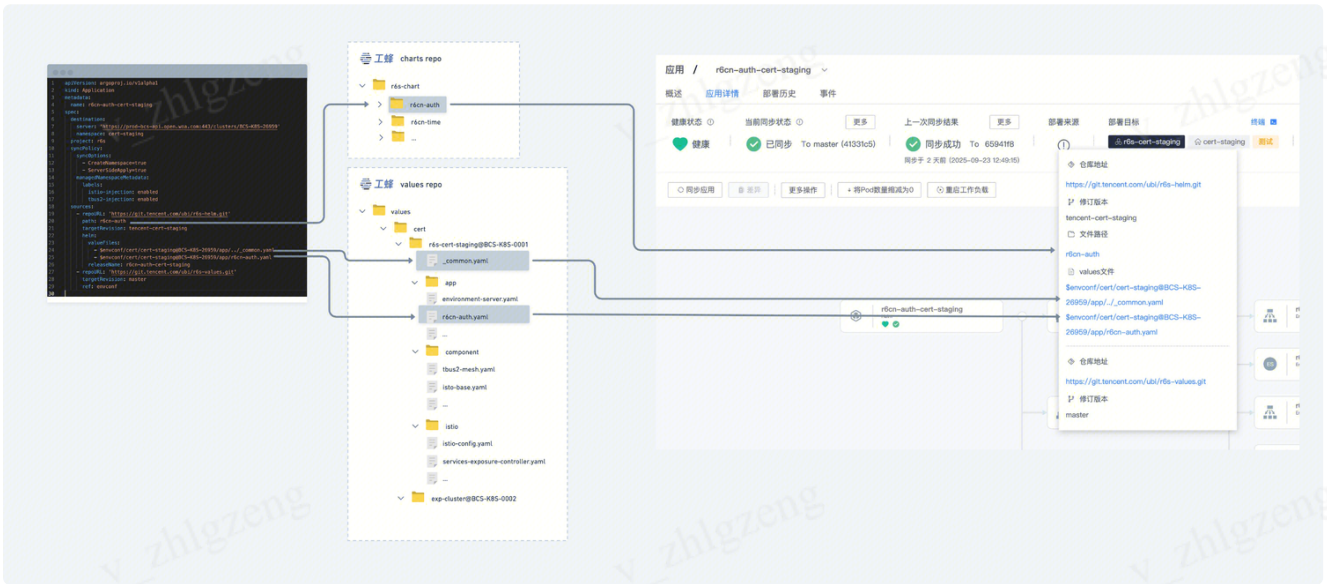
也就是说，一次上线不再只是“把镜像发到集群”，而是把这个服务依赖的一整套运行环境，通过代码完整描述出来。

六、实现方式：ApplicationSet + multi-source + Git Generator

6.1 Chart 与 values 分离

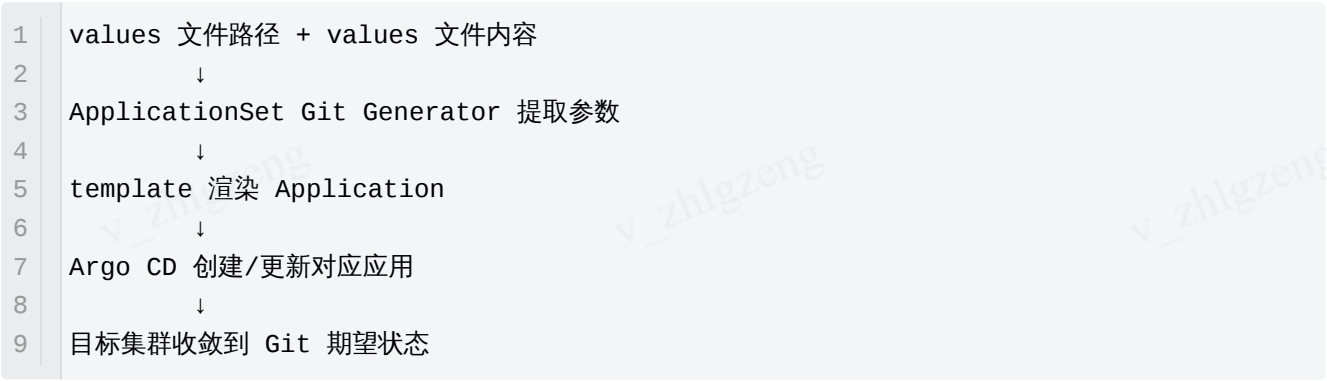
推荐把通用 Chart 与环境 values 分离：

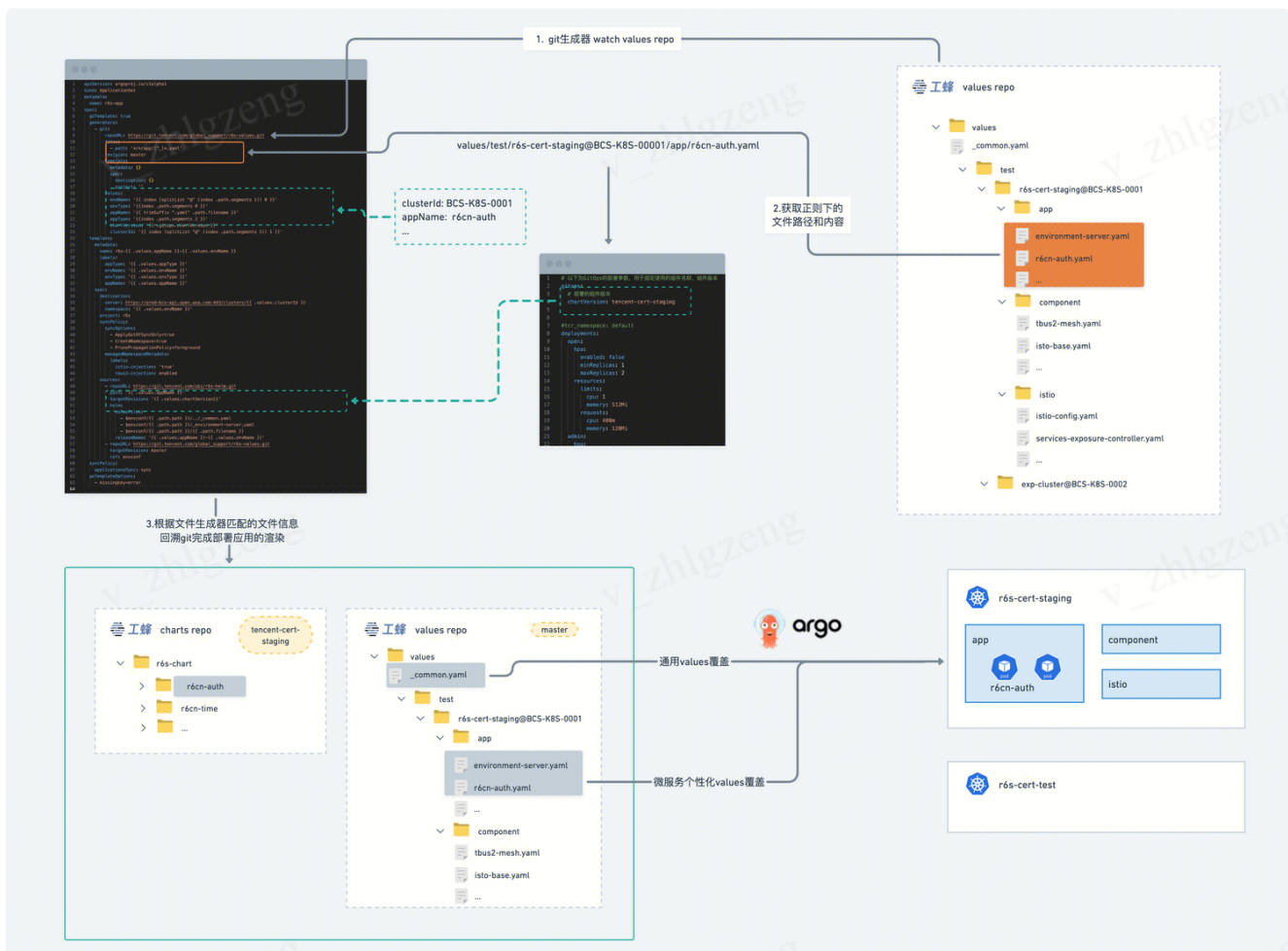
- Chart 仓库：描述应用通用结构，由研发/平台共同维护；
- Values 仓库：描述不同环境的参数差异，由 SRE/业务运维维护；
- ApplicationSet：通过 Git Generator 扫描 values 仓库，自动生成多个 Application。



6.2 ApplicationSet 自动发现环境文件

应用组根据 Git Generator 匹配 values 目录下的文件，结合 multi-source 渲染 Application：





示例：

```

1  apiVersion: argoproj.io/v1alpha1
2  kind: ApplicationSet
3  metadata:
4    name: demo-apps
5  spec:
6    goTemplate: true
7    generators:
8      - git:
9        repoURL: https://git.woa.com/demo/gitops-values.git
10       revision: prod
11       files:
12         - path: 'values-cn/prod/*/app/[^_]*.yaml'
13       values:
14         appName: '{{ trimSuffix ".yaml" .path.filename }}'
15         clusterId: '{{ index (splitList "@" (index .path.segments 2)) 1 }}'
16     template:
17       metadata:
18         name: demo-{{ .values.appName }}
19       spec:
20         project: demo
21         destination:
22           server: https://prod-bcs-api.open.woa.com:443/clusters/{{
23             .values.clusterId }}
24           namespace: '{{ .values.appName }}'
25         sources:
26           - repoURL: https://git.woa.com/demo/charts.git

```

```

26     path: app
27     targetRevision: '{{ .gitops.chartRevision }}'
28   helm:
29     valueFiles:
30       - $envconf/{{ .path.path }}/{{ .path.filename }}
31     - repoURL: https://git.woa.com/demo/gitops-values.git
32       targetRevision: prod
33       ref: envconf
34   syncPolicy:
35     applicationsSync: create-update
36     preserveResourcesOnDeletion: true

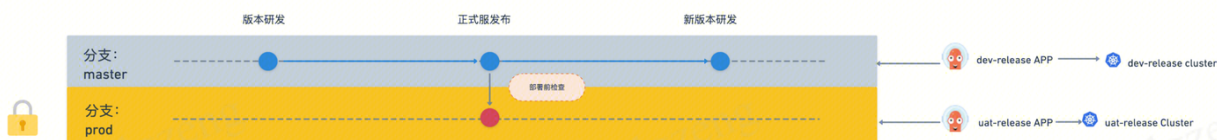
```

这个 YAML 表达的不是“一个部署动作”，而是一套可持续运行的环境生成规则。只要 Git 中新增 / 修改 values 文件，PowerAPP 就能按规则自动生成或更新对应应用。

七、版本发布与变更流程

生产环境最推荐的方式是：分支隔离 + MR 合并 + 分支保护 + 平台自动同步。

工蜂 r6s-deployments



推荐流程：

1. 开发 / 测试环境：允许更高频提交，快速验证配置；
2. 预发布环境：使用接近生产的 values 与基础设施规格；
3. 生产环境：ApplicationSet / Terraform Layer 监听 `prod` 或 `release/prod` 分支；
4. 生产分支开启保护：禁止直接 push，只允许 MR 合并；
5. MR 阶段自动检查：Helm template、K8s dry-run、Terraform plan、安全策略检查；
6. 合并后自动收敛：PowerAPP 监听 Git 变更并持续同步；
7. 异常时回滚 Git：回退 commit / tag，而不是手工逐项恢复控制台配置。

注意：每个分支保存该环境的完整期望状态。这意味着某个时间点的 Git commit 就是一份可复现的环境快照。

八、这种模式带来的核心收益

8.1 可追溯

所有变更都对应 commit / MR / reviewer / 合并时间。

当线上出问题时，不再问“谁改了控制台”，而是直接看 Git 历史：

- 1 哪个 MR 改了副本数？
- 2 哪个 commit 改了 CLB 监听器？

8.2 可复现

把某个环境恢复到历史状态，不再依赖人工记忆，只需回退到对应 commit。

- 1 Git commit = 环境快照
- 2 Git revert = 环境回滚

8.3 可评审

部署变更和基础设施变更都能在 MR 中被 Review：

- 镜像版本是否正确；
- 资源规格是否过大或过小；
- 是否误删了生产配置；
- Terraform plan 是否包含高风险 destroy；
- 安全组是否暴露高危端口。

8.4 可扩展

当业务从 1 个环境扩展到 10 个环境、从 1 个集群扩展到 100 个集群时，不需要复制大量平台配置，只需要维护目录结构和模板。

8.5 可治理

平台可以持续检测：

- 集群实际状态是否偏离 Git；
- 云资源是否发生漂移；
- 应用配置是否符合最佳实践；
- 高风险同步 / 删除 / destroy 是否被保护。

这让治理从“事后检查”变成“持续内建”。

8.6 活文档：Git 仓库就是业务运行态的说明书

很多团队都写过部署文档，但这类文档很容易变成“死文档”：上线时写了一版，后来环境迁移、参数调整、基础设施变更、临时修复不断发生，文档却没人同步维护。半年后再打开，里面的集群、域名、参数、截图可能早已过期。

部署即代码的价值在于：**Git 仓库本身就是业务的活文档。**

因为它不是旁路记录，而是直接驱动线上环境的事实来源：

- values 文件记录了每个环境的真实运行参数；
- 目录结构记录了环境、地域、集群、应用实例之间的映射；
- ApplicationSet 模板记录了批量交付规则；
- Terraform 代码记录了应用依赖的云资源；
- MR 讨论记录了为什么这样改、谁评审过、风险如何评估；
- commit 历史记录记录了业务运行态随时间演进的全过程。

这
和传统部署文档最大的区别是：

传统部署文档	GitOps 仓库活文档
写完后依赖人工维护	每次变更必须修改仓库，天然持续更新
描述“应该怎么部署”	直接定义“现在实际怎么部署”
很难确认是否过期	与线上状态持续对比，漂移会被发现
适合阅读，不参与执行	既可阅读，也直接驱动部署和回滚
变更原因散落在聊天/工单里	MR、commit、review 形成完整上下文

因此，新的同学接手业务时，不必到处问“这个服务到底部署在哪、依赖哪些资源、生产参数为什么这么配”。只要从 Git 仓库看起，就能获得一份**最新、可信、可执行**的业务运行说明书。

九、最佳实践

9.1 目录优先，分支辅助

推荐用目录表达环境、地域、集群、应用实例；用分支表达发布阶段或权限边界。

```
1 values-cn/prod/<cluster>/app/<app>.yaml
2 values-sg/prod/<cluster>/app/<app>.yaml
```

不要把所有环境差异都藏在分支里，否则全局拓扑不直观，跨环境对比成本高。

9.2 生产环境必须走 MR

生产分支必须开启保护：

- 禁止直接 push；
- 至少一名 reviewer；
- 关键应用建议双人 Review；
- Terraform destroy / Application 删除必须显式标注风险。

9.3 Chart 和 values 分离

Chart 关注“应用长什么样”，values 关注“环境如何不同”。

这样可以实现：

- 研发维护通用能力；
- SRE 管理环境差异；
- 同一套 Chart 复用于多个环境；
- values 仓库成为环境事实来源。

9.4 应用和基础设施分层，但流程统一

不要把所有 Terraform 资源和所有应用 values 混在一个大目录里。推荐按职责拆分，但都走同样的 GitOps 协作模式：

1	应用：Application / ApplicationSet
2	基础设施：Terraform Layer / LayerSet
3	统一流程：Git → MR → Diff/Plan → Review → Merge → Reconcile

9.5 高风险动作默认保守

- ApplicationSet 生产环境推荐 `applicationsSync: create-update` ；
- 开启 `preserveResourcesOnDeletion: true` ；
- Terraform Layer 生产环境推荐 `manual` 同步；
- 删除 Layer 前确认 `destroyResourcesOnDeletion` ；
- 高风险变更先看 diff / plan，再执行。

9.6 把仓库当成第一入口来维护

既然 Git 仓库是业务运行态的活文档，就应该有意识地提升它的可读性：

- 目录命名要能看出环境、地域、集群和应用；
- values 文件中的关键字段要有清晰命名；
- 公共配置与环境差异要拆分清楚；
- MR 描述要写明变更背景、影响范围、回滚方式；
- 对复杂资源依赖，可以在仓库中补充必要的 `README` 或注释，但不要让文档与实际配置分离。

十、常见误区

误区	正确理解
“部署即代码就是写 YAML”	YAML 只是载体，关键是 Git + Review + 自动检查 + 持续收敛
“GitOps 只管 K8s 应用”	PowerAPP 同时覆盖应用与 Terraform 基础设施
“平台点一下更快”	临时更快，但不可追溯、不可复现、不可审计，长期成本更高
“生产环境自动同步最先进”	生产更应重视保护：先 Review、看 Diff/Plan，再受控同步
“部署文档已经写了，就不需要代码化”	文档容易过期；GitOps 仓库既是事实来源，也是持续更新的活文档
“基础设施变更只能平台团队做”	可以由平台提供 Module，业务通过 MR 使用，既自助又受控

十一、可参考的 Demo

- ApplicationSet multi-source 示例：

<https://powerapp.woa.com/project/powerapp-demo/applicationset/powerapp-demo-mutisource-app/summary>

建议重点观察：

1. values 仓库如何通过目录表达环境和集群；
2. ApplicationSet 如何通过 Git Generator 自动发现应用；
3. chart 仓库与 values 仓库如何通过 multi-source 组合；
4. 分支保护与 MR 流程如何承载生产变更。

十二、相关文档

- [PowerAPP 介绍](#)
- [Application / 应用](#)
- [ApplicationSet / 应用组](#)
- [Infra GitOps 介绍](#)
- [Terraform Layer 功能介绍](#)
- [Terraform LayerSet 功能介绍](#)
- [应用组使用注意事项](#)
- [AI 小鲸](#)